

Утверждаю
Ректор университета
_____ О.Н. Федонин
« ____ » _____ 2025 г.

БЕЗОПАСНОСТЬ ОПЕРАЦИОННЫХ СИСТЕМ
МЕТОДИЧЕСКИЕ УКАЗАНИЯ ПО ВЫПОЛНЕНИЮ
КУРСОВОЙ РАБОТЫ

Методические указания
к выполнению курсовой работы

УДК 004.432.2

Безопасность операционных систем. Методические указания к выполнению курсовой работы [Электронный ресурс]: методические указания к выполнению курсовой работы. – Брянск: БГТУ, 2025. – 18 с.

Разработали:

В.А. Воронин, ст. преподаватель.

М.Ю. Рытов, канд. техн. наук., доц.

Рекомендовано кафедрой «Системы информационной безопасности» БГТУ (протокол №11 от 28.08.2025).

Научный редактор
Компьютерный набор

М.Ю. Рытов
В.А. Воронин

Методические указания публикуются в авторской редакции

Подписано в печать 28.08.17 г. Формат 60x84 1/16.
Усл. печ.л. 0,81 Уч.-изд.л. 0,81

Брянский государственный технический университет
Кафедра «Системы информационной безопасности», 58-83-54
241035, Брянск, бульвар 50 лет Октября, 7.

1. ЦЕЛЬ И ЗАДАЧИ КУРСОВОЙ РАБОТЫ

Курсовая работа по безопасности операционных систем выполняется для изучения устройства, принципов организации и взаимодействия подсистем защиты в ОС различной архитектуры, изучения защищенности заданных конфигураций различных операционных систем и приобретения навыков построения комплексной системы безопасности в заданной конфигурации операционной системы.

2. ТЕМАТИКА КУРСОВОЙ РАБОТЫ

Порядок функционирования систем безопасности ОС в общем случае определяется большим числом настроек параметров и элементов ее структуры, которые при инсталляции ОС по умолчанию устанавливаются таким образом, что решению задачи защиты информации в ОС отводится второстепенная роль. Данное обстоятельство вынуждает администратора ОС самостоятельно выбрать тот или иной путь решения задачи защиты информации.

3. ИСХОДНЫЕ ДАННЫЕ К КУРСОВОЙ РАБОТЕ

Обнаруживать, блокировать и предотвращать атаки можно несколькими путями. Первый, и самый распространенный, способ – это обнаружение уже реализуемых атак. Данный способ функционирует на втором этапе осуществления атаки. Этот способ применяется в "классических" системах обнаружения атак (например, RealSecure компании Internet Security Systems), межсетевых экранах и т.п. Однако, "недостаток" средств данного класса в том, что атаки могут быть реализованы повторно. Они также повторно обнаруживаются и блокируются. И так далее, до бесконечности. Второй путь – предотвратить атаки еще до их реализации. Осуществляется это путем поиска уязвимостей, которые могут быть использованы для реализации атаки. И, наконец, третий путь – обнаружение уже совершенных атак и предотвращение их повторного осуществления. Таким образом, системы обнаружения атак могут быть классифицированы по этапам осуществления атаки:

Системы, функционирующие на первом этапе осуществления атак и позволяющие обнаружить уязвимости информационной системы, используемые нарушителем для реализации атаки называются системами анализа защищенности (security assessment systems) или сканерами безопасности (security scanners). Обычно системы анализа защищенности не принято относить к классу средств обнаружения атак, однако, если

следовать описанным выше этапам осуществления атаки, то такое отнесение вполне логично.

Системы, функционирующие на втором этапе осуществления атаки и позволяющие обнаружить атаки в процессе их реализации, т.е. в режиме реального (или близкого к реальному) времени. Именно эти средства и принято считать системами обнаружения атак в классическом понимании. Помимо этого в последнее время выделяется новый класс средств обнаружения атак - обманные системы.

Системы, функционирующие на третьем этапе осуществления атаки и позволяющие обнаружить уже совершенные атаки. Эти системы делятся на два класса - системы контроля целостности, обнаруживающие изменения контролируемых ресурсов, и системы анализа журналов регистрации.

Сканер безопасности ОС состоит из двух основных компонентов. Первый производит анализ защищенности заданной конфигурации ОС, второй выполняет обнаружение реализуемых атак.

3.1. Анализ защищенности

Практически любой сканер проводит анализ защищенности в несколько этапов:

Сбор информации об узле. На данном этапе идентифицируются все активные на нем сервисы и демоны.

Обнаружение потенциальных уязвимостей. Сканер использует базу данных для сравнения собранных данных с известными уязвимостями при помощи проверки заголовков или активных зондирующих проверок. В некоторых системах все уязвимости ранжируются по степени риска. Например, в системе NetSonar уязвимости делятся на два класса: сетевые и локальные уязвимости. Сетевые уязвимости (например, воздействующие на маршрутизаторы) считаются более серьезными по сравнению с уязвимостями, характерными только для рабочих станций. Аналогичным образом “поступает” и Internet Scanner. Все уязвимости в нем делятся на три степени риска: высокая (High), средняя (Medium) и низкая (Low).

Подтверждение выбранных уязвимостей. Сканер использует специальные методы и моделирует (имитирует) определенные атаки для подтверждения факта наличия уязвимостей на выбранных узлах сети.

Генерация отчетов. На основе собранной информации система анализа защищенности создает отчеты, описывающие обнаруженные уязвимости. В некоторых системах (например, Internet Scanner и NetSonar) отчеты создаются для различных категорий пользователей, начиная от

администраторов сети и заканчивая руководством компании. Если первых в первую очередь интересуют технические детали, то для руководства компании необходимо представить красиво оформленные с применением графиков и диаграмм отчеты с минимумом подробностей. Немаловажным аспектом является наличие рекомендаций по устранению обнаруженных проблем. И здесь по праву лидером является система Internet Scanner, которая для каждой уязвимости содержит пошаговые инструкции для устранения уязвимостей, специфичные для каждой операционной системы. Во многих случаях отчеты также содержат ссылки на FTP- или Web-сервера, содержащие patch'и и hotfix'ы, устраняющие обнаруженные уязвимости.

Автоматическое устранение уязвимостей. Этот этап очень редко реализуется в сетевых сканерах, но широко применяется в системных сканерах (например, System Scanner). При этом данная возможность может реализовываться по-разному. Например, в System Scanner создается специальный сценарий (fix script), который администратор может запустить для устранения уязвимости. Одновременно с созданием этого сценария, создается и второй сценарий, отменяющий произведенные изменения. Это необходимо в том случае, если после устранения проблемы, нормальное функционирование узла было нарушено. В других системах возможности "отката" не существует.

Пример проверки, осуществляемой системой WebTrends Security Analyzer

```
<TestAuthor> WebTrends Corporation </TestAuthor>

<TestCopyright> Copyright 1998, WebTrends Corporation, All
Rights Reserved.
</TestCopyright>

<TestVersion> 2.0 </TestVersion>

=====

<TestDependency>estabvc</TestDependency>

<TestCategory>inventory</TestCategory>

=====

<TestTitle>Query OS Type via Netbios</TestTitle>

<TestVulnerabilityDescription>
```

This test attempts to determine the operating system type and version running on the specified hosts.
 </TestVulnerabilityDescription>

=====

<Test>

```
# osdetectnt.pl
# attempt to detect OS using a netbios over tcp/ip call

require "crowbar.pl";

$theTargetNetbiosName =
GetStringParam($crowbar::WTDB_NetbiosName);
crowbar::WTDebugOutput
    ("OSDetect -- the target netbios name is
$theTargetNetbiosName");

if($theTargetNetbiosName){
    $a = crowbar::WTGetNTOSInfo($theTargetNetbiosName);
    if($a){
        $a =~ /^OSTYPE (.*) :VERSION (.*)/;
        $type = $1;
        $version = $2;
        crowbar::WTDebugOutput("Type is $type, version is
$version\n");
        if($version =~ m/OSVersion_Unknown/){
            crowbar::WTAddRecord( $crowbar::WTDB_OSVersion,
                length("Unknown") + 1,
                "Unknown", -1);
        }
        elseif($version =~ m/OSVersion_WindowsNT_3_5_0/){
            crowbar::WTAddRecord( $crowbar::WTDB_OSVersion,
                length("Version 3.5") + 1,
                "Version 3.5", -1);
        }
        elseif($version =~ m/OSVersion_WindowsNT_3_5_1/){
            crowbar::WTAddRecord( $crowbar::WTDB_OSVersion,
                length("Version 3.51") + 1,
                "Version 3.51", -1);
        }
        elseif($version =~ m/OSVersion_WindowsNT_4_0/){
            crowbar::WTAddRecord( $crowbar::WTDB_OSVersion,
                length("Version 4.0") + 1,
                "Version 4.0", -1);
        }
        elseif($version =~ m/OSVersion_WindowsNT_5_0/){
            crowbar::WTAddRecord( $crowbar::WTDB_OSVersion,
                length("Version 5.0") + 1,
                "Version 5.0", -1);
        }
    }
}
```

```

}

if($type =~ m/OSType_Unknown/){
crowbar::WTAddRecord( $crowbar::WTDB_OSType,
    length("Unknown") + 1,
    "Unknown", -1);
}
elseif($type =~ m/OSType_Unix/){
crowbar::WTAddRecord( $crowbar::WTDB_OSType,
    length("Unix Server") + 1,
    "Unix Server", -1);
}
elseif($type =~ m/OSType_WindowsNTServer/){
crowbar::WTAddRecord( $crowbar::WTDB_OSType,
    length("Windows NT Server") + 1,
    "Windows NT Server", -1);
}
elseif($type =~ m/OSType_WindowsNTPDC/){
crowbar::WTAddRecord( $crowbar::WTDB_OSType,
    length("Windows NT Primary Domain Controller") +
1,
    "Windows NT Primary Domain Controller", -1);
}
elseif($type =~ m/OSType_WindowsNTBDC/){
crowbar::WTAddRecord( $crowbar::WTDB_OSType,
    length("Windows NT Backup Domain Controller") + 1,
    "Windows NT Backup Domain Controller", -1);
}
elseif($type =~ m/OSType_WindowsNTWorkstation/){
crowbar::WTAddRecord( $crowbar::WTDB_OSType,
    length("Windows NT Workstation") + 1,
    "Windows NT Workstation", -1);
}
elseif($type =~ m/OSType_WindowsNT/){
crowbar::WTAddRecord( $crowbar::WTDB_OSType,
    length("Windows NT") + 1,
    "Windows NT", -1);
}
elseif($type =~ m/OSType_Windows95/){
crowbar::WTAddRecord( $crowbar::WTDB_OSType,
    length("Windows 95/98") + 1,
    "Windows 95/98", -1);
}
elseif($type =~ m/OSType_Windows98/){
crowbar::WTAddRecord( $crowbar::WTDB_OSType,
    length("Windows 98") + 1,
    "Windows 98", -1);
}
}
}
}
</Test>

```

Пример проверки, осуществляемой системой CyberCop CASL
spoof_check.cape

```
# this script is used by the built-in filter checks
# please do not modify it
ip
ip_version=4
ip_proto=IPPROTO_UDP
ip_flags=0
ip_id=42
ip_done

udp
udp_sport=6834
udp_dport=5574
udp_done

data=SAS-ipspoofing

end_of_packet
```

3.2. Обнаружение атак

Основные компоненты модуля обнаружения атак:

Подсистема управления компонентами

Данная подсистема позволяет управлять различными компонентами системы обнаружения атак. Управление может осуществляться, как при помощи внутренних протоколов и интерфейсов, так и при помощи уже разработанных стандартов, например, SNMP. Под термином "управление" понимается как возможность изменения политики безопасности для различных компонентов системы обнаружения атак (например, модулей слежения), так и получение информации от этих компонент (например, сведения о зарегистрированной атаке).

Подсистема обнаружения атак

Основной компонент системы обнаружения атак, который осуществляет анализ информации, получаемой от модуля слежения. По результатам анализа данная подсистема может идентифицировать атаки, принимать решения относительно вариантов реагирования, сохранять сведения об атаке в хранилище данных и т.д.

Подсистема реагирования

Подсистема, осуществляющая реагирование на обнаруженные атаки и иные контролируемые события.

Модуль слежения

Компонент, обеспечивающий сбор данных из контролируемого пространства (журнала регистрации или сетевого трафика). У разных производителей называется также сенсором (sensor), монитором (monitor), зондом (probe) и т.д. В зависимости от архитектуры построения системы

обнаружения атак может быть физически отделен (архитектура "агент-менеджер") от других компонентов, т.е. находиться на другом компьютере.

База знаний

В зависимости от методов, используемых в системе обнаружения атак, база знаний может содержать профили пользователей и вычислительной системы, сигнатуры атак или подозрительные строки, характеризующие несанкционированную деятельность. Эта база может пополняться производителем системы обнаружения атак, пользователем системы или третьей стороной, например, аутсорсинговой компанией, осуществляющей поддержку этой системы.

Хранилище данных

Обеспечивает хранение данных, собранных в процессе функционирования системы обнаружения атак.

Основными механизмами обнаружения атак являются

– Анализ журналов регистрации.

Этот один из самых первых реализованных методов обнаружения атак. Он заключается в анализе журналов регистрации (log, audit trail), создаваемых операционной системой, прикладным программным обеспечением, маршрутизаторами и т.д. Записи журнала регистрации анализируются и интерпретируются системой обнаружения атак.

К достоинствам этого метода относится простота его реализации. Однако за этой простотой скрывается немало недостатков. Во-первых, для достоверного обнаружения той или иной подозрительной деятельности необходима регистрация в журналах большого объема данных, что отрицательно сказывается на скорости работы контролируемой системы. Во-вторых, при анализе журналов регистрации очень трудно обойтись без помощи специалистов, что существенно снижает круг распространения этого метода. В-третьих, до настоящего момента нет унифицированного формата хранения журналов. И хотя работы в этой области ведутся, например, в лаборатории COAST или в компании WebTrends, до их завершения еще очень далеко. И, наконец, анализ уже записанных в журнал регистрации записей говорит о том, что анализ осуществляется не в реальном режиме времени.

Как правило, анализ журналов регистрации является дополнением к другим методам обнаружения атак, в частности, к обнаружению атак "на лету". Использование этого метода позволяет проводить "разбор полетов" уже после того, как была зафиксирована атака, для того чтобы выработать

эффективные меры предотвращения аналогичных атак в будущем.

– Анализ "на лету"

Этот метод заключается в мониторинге сетевого трафика в реальном или близком к реальному времени и использовании соответствующих алгоритмов обнаружения. Очень часто используется механизм поиска в трафике определенных строк, которые могут характеризовать несанкционированную деятельность. К таким строкам можно отнести '\\WINNT\\SYSTEM32\\CONFIG' (данная строка описывает путь к файлам Sam, Security и т.д.) или '/etc/passwd' (данная строка описывает путь к списку паролей ОС Unix).

Использование метода обнаружения атак в сетевом трафике дает два основных преимущества. Во-первых, один агент системы обнаружения атак может просматривать целый сегмент сети с многочисленными хостами, в то время как, для предыдущего метода необходимо на каждый анализируемый узел устанавливать свой агент. Этот метод позволяет обнаруживать атаки против всех элементов сети предприятия, начиная от атак на маршрутизаторы и заканчивая атаками на прикладные приложения. Во-вторых, системы, построенные с учетом этого метода, могут определять атаки в реальном масштабе времени и останавливать атаки до достижения ими цели.

– Использование профилей "нормального" поведения

Профили нормального поведения используются для наблюдения за пользователями, системной деятельностью или сетевым трафиком. Данные наблюдения сравниваются с ожидаемыми значениями профиля нормального поведения, который строится в период обучения системы обнаружения атак.

Этот метод редко используется в современных системах защиты информации (хотя такие попытки и делаются). Использование профилей нашло свое практическое применение в системах обнаружения мошенничества (fraud detection systems), используемых в финансовых структурах или у операторов связи.

– Использование сигнатур атак

Данный метод очень часто сопоставляют с анализом "на лету". Метод заключается в описании атаки в виде сигнатуры (signature) и поиска данной сигнатуры в контролируемом пространстве (сетевом трафике, журнале регистрации и т.д.). В качестве сигнатуры атаки может выступать шаблон действий или строка символов, характеризующие аномальную деятельность. Эти сигнатуры хранятся в базе данных, аналогичной той,

которая используется в антивирусных системах. Собственно говоря, антивирусные резидентные мониторы являются частным случаем системы обнаружения атак, но т.к. эти направления изначально развивались параллельно, то принято разделять их.

4. ПОЛУЧЕНИЕ НУЖНОЙ ИНФОРМАЦИИ

Получить нужную информацию можно обратившись к консоли или конфигурационным файлам. В разрабатываемом ПО получены текстовые данные необходимо обработать и представить в легко читаемом, для пользователя, виде. Далее разберем как полечит необходимые данные, на примере Ubuntu 20.04.

1. Вид операционной системы.

Можно использовать следующие команды:

lsb_release -a

Вывод:

No LSB modules are available.

Distributor ID: Ubuntu

Description: Ubuntu 20.04.4 LTS

Release: 20.04

Codename: focal

uname -r

Вывод:

5.13.0-39-generic

2. Определение имени узла, рабочей группы или домена

Можно использовать следующейю команду:

hostnamectl

Вывод:

Static hostname: Home-PC

Icon name: computer-desktop

Chassis: desktop

*Machine ID: ******

*Boot ID: ******

Operating System: Ubuntu 20.04.4 LTS

Kernel: Linux 5.13.0-39-generic

Architecture: x86-64

Также информацию можно найти в файле: **/etc/samba/smb.conf**

Содержание:

.....

[global]

```
## Browsing/Identification ###
# Change this to the workgroup/NT-domain name your Samba server will part of
  workgroup = WORKGROUP
# server string is the equivalent of the NT Description field
  server string = %h server (Samba, Ubuntu)
##### Networking #####
.....
```

3. Установленные обновления безопасности

Можно использовать следующей командой:

```
apt list --upgradeable --installed
```

Вывод:

Вывод списка... Готово

```
accountsservice/focal-updates,focal-security,now 0.6.55-0ubuntu12~20.04.5 amd64
```

```
  [установлен, автоматически]
```

```
acl/focal,now 2.2.53-6 amd64 [установлен, автоматически]
```

```
.....
```

```
zlib1g/focal-updates,focal-security,now 1:1.2.11.dfsg-2ubuntu1.3 i386 [установлен]
```

```
zstd/focal-updates,focal-security,now 1.4.4+dfsg-3ubuntu0.1 amd64 [установлен]
```

Далее, в данном случае, нужно выбрать только строки содержащие «focal-security».

4. Учетные записи администраторов

Информацию можно найти в файле: **/etc/group**

Содержание:

```
root:x:0:
```

```
daemon:x:1:
```

```
bin:x:2:
```

```
sys:x:3:
```

```
adm:x:4:syslog,ferix
```

```
.....
```

```
sudo:x:27:ferix
```

```
.....
```

В данном случае администраторами являются пользователи находящиеся в группах «adm» и «sudo», а также пользователь «root».

Дополнительную информацию можно найти в файле: **/etc/passwd**

Вывод:

```
root:x:0:0:root:/root:/bin/bash
```

```
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

```
.....
```

```
syslog:x:104:110::/home/syslog:/usr/sbin/nologin
```

```
_apt:x:105:65534::/nonexistent:/usr/sbin/nologin
```

```
.....
fenix:x:1000:1000:Владимир,,,:/home/fenix:/bin/bash
systemd-coredump:x:999:999:systemd Core Dumper:/:usr/sbin/nologin
.....
```

Обратите внимание на параметр оболочки пользователя, если он задан в значение «/usr/sbin/nologin», такой пользователь не может войти в локальную систему и следовательно не является локальным администратором.

5. Политика паролей.

Конфигурация парольной политики зависит от установленной библиотеки `libpam_pwquality`. Проверим её наличие.

apt-cache policy *pam-pwquality

Вывод:

libpam-pwquality:

Установлен: (отсутствует)

Кандидат: 1.4.2-1build1

Таблица версий:

1.4.2-1build1 500

500 [http://mirror.yandex.ru/ubuntu focal/main amd64 Packages](http://mirror.yandex.ru/ubuntu/focal/main/amd64/Packages)

Библиотека отсутствует, политика проверки сложности пароля не применяется. Если библиотека установлена информацию о парольной политике можно найти в файле **/etc/pam.d/common-password**.

Другие параметра можно узнать из файла: **gedit /etc/login.defs**

Содержание:

```
.....
PASS_MAX_DAYS 99999
```

```
PASS_MIN_DAYS 0
```

```
PASS_WARN_AGE 7
.....
```

```
ENCRYPT_METHOD SHA512
.....
```

Как видно из параметров: максимально время жизни пароля = 99999; минимальное время жизни пароля = 0; количество дней предупреждения до истечения срока действия пароля = 7; алгоритм для шифрования пароля = SHA512.

6. Аудит системы

Для проведения аудита безопасности, в Linux системах, можно в ручную проанализировать системные параметры путем выполнения

специальных команд и анализа конфигурационных файлов. Если перед вами не стоит задача анализа определенного набора параметров более легкий способ, это воспользоваться специально разработанным программным обеспечением. Для Linux ОС подобного ПО большое множество: lynis, auditd, lunar, nix auditor, и др.

Можно использовать следующей командой:

sudo lynis audit system

Вывод:

```

.....
[+] Initializing program
-----
- Detecting OS...                [ DONE ]
- Checking profiles...          [ DONE ]
- Detecting language and localization [ ru ]
-----
Program version:      3.0.7
Operating system:     Linux
Operating system name: Ubuntu
Operating system version: 20.04
Kernel version:       5.13.0
Hardware platform:    x86_64
Hostname:             Home-PC
-----
Profiles:             /etc/lynis/default.prf
Log file:             /var/log/lynis.log
Report file:          /var/log/lynis-report.dat
Report version:       1.0
Plugin directory:     /usr/share/lynis/plugins
-----
Auditor:              [Not Specified]
Language:             ru
Test category:        all
Test group:           all
-----
- Program update status...      [ ОБНОВЛЕНИЙ НЕТ ]
.....

```

Чтобы использовать подобные утилиты в разрабатываемом вами ПО, необходимо чтобы они устанавливались вместе.

13. Анализ журналов («логов»).

В данной ОС основной способ получить сводную информацию из системных журналов использование команды «journalctl» с параметрами.

5. ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

При выполнении курсовой работы студент должен разработать программу-анализатор заданной конфигурации определенной ОС. Разработанная программа в соответствии с заданными параметрами (см. табл. 1) должна оценить безопасность конфигурации ОС и указать пути ее улучшения.

Задание на проектирование выдается студенту в течение первых двух недель четвертого семестра. Задание оформляется на типовом бланке отдельно на каждый проект, включая индивидуальные задания. На бланке типового задания указывается тема работы “Сканер безопасности ОС” и исходные данные, определенные по номеру варианта, заданному преподавателем. В течение первых четырех недель с момента выдачи задания исходные данные могут быть откорректированы по согласованию с руководителем работы. Индивидуальное задание должно быть согласовано с ведущим лектором курса “Безопасность операционных систем”.

В ходе выполнения курсовой работы необходимо разработать ПО, которое автоматически собирает и предоставляет пользователю следующие информацию:

1. Вид операционной системы. (например для Linux систем: версия дистрибутива — Ubuntu 20.04.4 LTS, кодовое имя — focal, версия ядра — 5.13.0-39-generic).
2. Определение имени узла, рабочей группы или домена (NetBIOS), (например: имя компьютера — Home-PC, класс — рабочая станция, рабочая группа — WORKGROUP).
3. Установленные обновления безопасности (например для Windows из базы Microsoft или заданные заранее).
4. Учетные записи администраторов (перечень администраторов, в локальной группе администраторов, имеющих высокие полномочия).
5. Политика паролей. (Проверка на простые пароли, слабые пароли, срок действия паролей, минимальная длина паролей, система шифрования пароля).
6. Аудит системы (какие параметры безопасна/небезопасны).
7. Сетевые настройки (TCP IP и т.д.)
8. Открытые ресурсы (совместно используемые ресурсы, разделяемые ресурсы NetBIOS (NetBIOS Share))

9. Запущенные сервисы (какие необходимы в различных случаях, особенно сетевые, анализ редко используемых сетевых сервисов)
10. Файловая система (Тип файловой системы, определить права на доступ к особо важным файлам, разрешения на основные файлы и папки)
11. Реестр. (Разрешения для реестра, ключи).
12. Групповые политики
13. Дополнительные параметры (шифрование SAM, агент восстановления, гостевой вход, кэширование паролей).
14. Анализ журналов («логов»). Нужно вывести содержание основных журналов с возможностями сортировки по времени, пользователям, сервису и критичности событий. Также необходимо выявить периодичность повторяющихся событий.
15. Обнаружение сканирования портов

6. ОБЪЕМ И ОФОРМЛЕНИЕ КУРСОВОЙ РАБОТЫ

Курсовая работа состоит из пояснительной записки и информационно-программных средств, реализующих задание на курсовое проектирование.

Пояснительная записка (ПЗ) выполняется на листах белой бумаги формата А4 (210х297 мм) машинописным или рукописным способом, оформляется согласно требованиям ЕСПД и должна содержать: титульный лист; бланк задания; введение; основное содержание (1 часть – общая постановка задачи и пути ее решения, 2 часть – решение в общем виде, 3 часть – назначение отдельных модулей и их взаимодействие, 4 часть – контрольный пример); заключение; библиографический список; приложение, содержащее фрагменты программ и изображения входных и выходных документов, подтверждающие выполнение основных этапов курсовой работы и отражающие ее отличия от других вариантов заданий.

Работающие варианты программ вместе с исходными текстами предоставляются преподавателю в электронном виде.

7. ЗАЩИТА КУРСОВОЙ РАБОТЫ

Выполненная и оформленная полностью курсовая работа предоставляется руководителю на проверку, который после проверки пояснительной записки и информационно-программного обеспечения подписывает ее к защите или возвращает студенту на доработку в

зависимости от готовности работы.

Защищается курсовая работа перед комиссией из двух-трех преподавателей кафедры.

Защита работы заключается в кратком докладе студента об основных проблемах, возникших при разработке информационно-программного обеспечения, принятых им способах их решения и полученных результатах, и ответа им на вопросы членов комиссии, с демонстрацией работы программы.

Комиссия оценивает защиту, как по качеству выполненной работы (эффективности работы программы, соответствия ее заданию), так и по уровню знаний студента, проявленных им в процессе защиты.

В случае неудовлетворительной оценки студент получает новое задание и выполняет работу заново.

СПИСОК РЕКОМЕНДУЕМОЙ ЛИТЕРАТУРЫ

1. Райкин И.Л. Информационная безопасность и защита информации: учебник для вузов / НГТУ им. Р.Е. Алексеева. – Нижний Новгород, 2011. – 256 с.
2. Зегжда Д.П., Ивашко А.М. Основы безопасности информационных систем. М.: Горячая линия – Телеком, 2000.
3. Безбогов, А.А. Безопасность операционных систем: учебное пособие / А.А. Безбогов, А.В. Яковлев, Ю.Ф. Мартемьянов. – М. : "Издательство Машиностроение-1", 2007. – 220 с.
4. Проскурин В. Г. Защита в операционных системах. Учебное пособие для вузов. -М.: Горячая линия - Телеком, 2014. - 192 с: ил.
5. Дейтел, Х.М. Операционные системы. Ч. 2: Распределенные системы, сети, безопасность / Х.М. Дейтел, П.Дж. Дей-тел, Д.Р. Чофнес. – М. : Бином, 2006.
6. Дейтел, Х.М. Операционные системы. Ч. 1: Основы и принципы / Х.М. Дейтел, П.Дж. Дейтел, Д.Р. Чофнес. – М. : Бином, 2007.
7. Гордеев, А.В. Операционные системы : учебник для вузов / А.В. Гордеев. – СПб. : Питер, 2008. – 416 с.
8. Олифер, В.Г. Сетевые операционные системы / В.Г. Олифер, Н.А. Олифер. – СПб. : Питер, 2007. – 544 с.
9. Танненбаум, Э. Современные операционные системы. 2-е изд. / Э. Танненбаум. – СПб. : Питер, 2006. – 1040 с.
10. Жигулин Г.П. Организационное и правовое обеспечение информационной безопасности [Электронный ресурс]: учебное пособие / Г.П. Жигулин. —Электрон. текстовые данные. —СПб.: Университет ИТМО, 2014. —174с. —2227-8397. —Режим доступа: <http://www.iprbookshop.ru/67451.html>
11. Шаньгин В.Ф. Информационная безопасность компьютерных систем и сетей: учеб. пособие для сред. проф. образования. -М.: ФОРУМ: ИНФРА-М, 2016. -415с.